

Implementación de un Vision Transformer desde cero en CUDA: impacto del número de parches y de la jerarquía de memoria

Wilson Ramos Pacco, Jhonatan Usccs Giraldo Bilbao,
Alvaro Sergio Cano Luque, Randú Jean Franco Cerpa García

Tópicos en Inteligencia Artificial — Universidad Nacional de San Agustín

Repositorio: <https://github.com/WilsonRamos/VisTransformer>

30 de julio de 2026

1. Contexto

La visión por computadora estuvo dominada durante casi tres décadas por redes convolucionales [6], cuya arquitectura codifica de fábrica el sesgo inductivo de localidad: un píxel se relaciona principalmente con sus vecinos inmediatos, aprendida con filtros compartidos que recorren toda la imagen. Dosovitskiy et al. [3] mostraron que ese sesgo no es indispensable: un Transformer [9] sin ninguna noción incorporada de vecindad espacial, alimentado con parches de imagen como secuencia de tokens, iguala o supera a las CNN con datos suficientes. La contrapartida es que el modelo debe *aprender* desde cero qué parches están relacionados, tarea que la convolución resuelve por diseño; esta ausencia de sesgo inductivo es precisamente el mecanismo que la Sección 5.1 invoca para explicar por qué, con pocos datos, una secuencia más corta resulta más fácil de optimizar.

Sobre el tamaño de parche, el propio trabajo de Dosovitskiy et al. reporta un compromiso contrario al que estudia este trabajo: sobre ImageNet con preentrenamiento a gran escala, reducir el tamaño de parche (más tokens) generalmente mejora la precisión, porque el detalle espacial adicional sí aporta información útil cuando sobran datos e imágenes complejas para aprovecharlo. Este trabajo se ubica en el extremo opuesto: imágenes de 28×28 muy simples y apenas 5 000 ejemplos de entrenamiento. Ese contraste motiva la lectura de la Sección 6: el compromiso entre tokens y precisión podría no ser una propiedad fija del Transformer, sino depender de cuánta información discriminativa hay realmente para extraer a mayor resolución.

Sobre la jerarquía de memoria, la atención estándar materializa una matriz de *scores* de $T \times T$, cuyo acceso repetido desde memoria global es exactamente el cuello de botella que ataca el *tiling* en memoria compartida, siguiendo el patrón clásico de multiplicación de matrices por bloques [5]. FlashAttention [2] lleva la misma idea más lejos: funde *scores*, softmax y la multiplicación por V en un único kernel que nunca escribe la matriz $T \times T$ en memoria global, evitando el problema en vez de solo mitigarlo. El experimento de memoria de este trabajo (Sección 5.2) estudia la misma palanca a una escala mucho más pequeña y con los dos kernels sin fusionar, lo que permite aislar el efecto de cada uno por separado; el hallazgo de que el beneficio del tiling no es uniforme entre ambos es, en cierto sentido, una motivación adicional para el enfoque fusionado que usa FlashAttention.

Alcance. Este trabajo se limita a MNIST, a una arquitectura fija de dos bloques con $D = 64$, a tres tamaños de parche específicos y a una sola semilla aleatoria por configuración. No evalúa en imágenes naturales de mayor resolución, no compara contra una implementación fusionada al estilo FlashAttention, y no repite las corridas para estimar variabilidad (ver Limitaciones, Sección 6). Dentro de ese alcance, el objetivo es aislar y explicar dos efectos concretos, no producir un modelo competitivo con el estado del arte.

2. Problema

Los Vision Transformers (ViT) [3] adaptan la arquitectura Transformer [9] al dominio visual mediante una idea sencilla: una imagen se corta en parches cuadrados no solapados, cada parche se proyecta linealmente a un vector de dimensión D , y la secuencia resultante se procesa con auto-atención exactamente igual que una secuencia de palabras. Esa decisión de diseño (*cuán grandes son los parches*) gobierna todo lo demás: fija el número de tokens T , y con él tanto el detalle espacial que el modelo puede representar como el coste computacional, porque la auto-atención escala como $O(T^2)$.

El problema que aborda este trabajo tiene dos caras. La primera es de aprendizaje profundo: **¿qué tamaño de parche conviene para clasificar dígitos de MNIST [6], y cuál es el compromiso real entre precisión y coste?** La segunda es de programación paralela: los productos matriciales de la atención son el núcleo computacional del modelo, y su rendimiento en GPU depende críticamente de cómo se usa la jerarquía de memoria. **¿Cuánto se gana al pasar de un kernel que lee todo de memoria global a uno que reutiliza los datos mediante tiling en memoria compartida, y cómo depende esa ganancia del número de tokens?**

Ambas preguntas se conectan: cuantos más parches, mayor es la matriz de atención $T \times T$, y mayor debería ser el beneficio del tiling. Para poder responderlas con propiedad, el modelo se implementa **íntegramente con kernels CUDA propios**, incluido el paso hacia atrás, derivado y programado a mano capa por capa, sin recurrir a cuDNN ni a cuBLAS. Escribir la retropropagación a mano no es un capricho: es la única forma de ver explícitamente cómo circula el gradiente por LayerNorm, por el softmax de la atención y por cada proyección lineal.

3. Objetivo

Objetivo general. Implementar, entrenar y evaluar un Vision Transformer completo sobre MNIST usando exclusivamente kernels CUDA escritos desde cero, y caracterizar experimentalmente el efecto del tamaño de parche y de la jerarquía de memoria sobre la precisión y el rendimiento.

Objetivos específicos.

1. Programar los kernels de propagación hacia adelante y hacia atrás de todas las capas del ViT (patch embedding, atención multi-cabeza, LayerNorm, MLP con GELU y cabeza de clasificación), con gradientes analíticos cerrados y sin diferenciación automática.
2. Verificar la corrección de la implementación de forma independiente, contrastando cada gradiente analítico contra una aproximación numérica y la arquitectura completa contra una referencia en PyTorch.
3. Medir precisión, tiempo de entrenamiento, tiempo de inferencia, uso de memoria y número de parámetros para parches de 4×4 , 7×7 y 14×14 , y explicar el compromiso resultante.
4. Implementar dos versiones del kernel crítico de la atención: una basada solo en memoria global y otra con tiling en memoria compartida. Medir su tiempo de forma aislada con `cudaEvent_t` y cuantificar el *speedup* en función del número de tokens.

4. Metodología

4.1. Datos

Se usan los archivos originales de MNIST en formato IDX. El cargador (`src/data_loader.c`) interpreta la cabecera respetando el orden de bytes *big-endian* del formato, valida el número mágico

(0x00000803 para imágenes, 0x00000801 para etiquetas), normaliza los píxeles a $[0, 1]$ y deja las imágenes en un bloque contiguo de `float` listo para una única transferencia a GPU. Antes de entrenar se ejecuta un test que dibuja dígitos como *ASCII art* y comprueba que la media de los píxeles sea 0,1307, el valor conocido de MNIST normalizado; una media distinta delataría un error de *stride* o de cabecera.

Por presupuesto de tiempo se emplea un subconjunto de 5 000 imágenes de entrenamiento y 2 000 de test, configurable con las opciones `--train-n` y `--test-n`.

4.2. Arquitectura

El modelo es un ViT reducido, deliberadamente pequeño para que las seis corridas quepan en el presupuesto de cómputo:

Dimensión de embedding D	64
Cabezas de atención H	4 (dimensión por cabeza $D_h = 16$)
Bloques Transformer	2, en configuración <i>pre-norm</i>
Oculto del MLP	128 ($2D$)
Tokens T	49+1, 16+1 o 4+1 según el parche
Parámetros	$\approx 72\,074$

La imagen 28×28 se divide en parches no solapados; los tres tamaños elegidos (4, 7 y 14) son precisamente los divisores de 28 que dan particiones limpias. Cada parche se aplan a p^2 valores y se proyecta a D dimensiones. Se antepone un token [CLS] aprendible, cuya representación final alimenta la cabeza de clasificación, y se suman **embeddings posicionales aprendibles**. Se prefirieron estos a los sinusoidales fijos porque la longitud de secuencia es constante dentro de cada configuración y muy corta (entre 5 y 50 posiciones): la tabla aprendible cuesta apenas $T \cdot D$ parámetros y evita justificar una elección arbitraria de frecuencias, pensada para secuencias de longitud variable.

Cada bloque aplica $x \leftarrow x + \text{Attn}(\text{LN}(x))$ y luego $x \leftarrow x + \text{MLP}(\text{LN}(x))$. La variante *pre-norm* [10] normaliza antes de cada submódulo y estabiliza el entrenamiento sin necesidad de *warmup* del learning rate, algo importante con un presupuesto de solo 15 épocas. La activación del MLP es GELU [4] en su aproximación con tangente hiperbólica.

4.3. Retropropagación analítica

Todo el paso hacia atrás está derivado a mano. Para una capa lineal $Y = XW + b$ con $X \in \mathbb{R}^{M \times K}$ y $W \in \mathbb{R}^{K \times N}$ se usan las tres identidades

$$\frac{\partial \mathcal{L}}{\partial X} = \frac{\partial \mathcal{L}}{\partial Y} W^T, \quad \frac{\partial \mathcal{L}}{\partial W} = X^T \frac{\partial \mathcal{L}}{\partial Y}, \quad \frac{\partial \mathcal{L}}{\partial b_n} = \sum_m \frac{\partial \mathcal{L}}{\partial Y_{mn}}, \quad (1)$$

que corresponden exactamente a los tres kernels `gemm_nt`, `gemm_tn` y `bias_backward`.

Para LayerNorm [1], con $\hat{x} = (x - \mu)/\sigma$, $r = 1/\sigma$ y g el parámetro de escala, los términos que pasan por μ y por σ se agrupan hasta llegar a la forma compacta que implementa el kernel:

$$\frac{\partial \mathcal{L}}{\partial x_j} = r (\delta_j - \bar{\delta} - \hat{x}_j \bar{\delta \hat{x}}), \quad \delta_j = \frac{\partial \mathcal{L}}{\partial y_j} g_j, \quad (2)$$

donde las barras denotan promedios sobre la dimensión normalizada. Es decir, dos productos escalares por fila, resueltos con reducciones en memoria compartida.

Para el softmax por filas de la atención, con $p = \text{softmax}(s)$, la jacobiana $\partial p_i / \partial s_j = p_i (\delta_{ij} - p_j)$ colapsa en

$$\frac{\partial \mathcal{L}}{\partial s_j} = p_j \left(\frac{\partial \mathcal{L}}{\partial p_j} - \sum_k p_k \frac{\partial \mathcal{L}}{\partial p_k} \right). \quad (3)$$

Finalmente, softmax y entropía cruzada se derivan de forma conjunta, lo que cancela términos y deja simplemente $\partial\mathcal{L}/\partial\text{logits} = (p - \mathbf{1}_y)/B$, una expresión mucho más estable numéricamente que encadenar ambas por separado.

El optimizador es AdamW [7] implementado como un único kernel elemental sobre todo el bloque contiguo de parámetros. Se eligió Adam sobre SGD porque los Transformers son notoriamente sensibles al learning rate con SGD y con 15 épocas no hay margen para ajustarlo.

4.4. Diseño de los kernels

Los productos matriciales usan bloques de $16 \times 16 = 256$ hilos, múltiplo del tamaño de warp y con buena ocupación en una T4 [8]. Cada bloque calcula un tile de 16×16 de la salida; el recorrido de la dimensión contraída se hace por tiles, trayendo cooperativamente un fragmento de cada operando a memoria compartida [5]. Los arreglos compartidos llevan una columna de relleno ([16] [17]) para romper la alineación con los 32 bancos de memoria compartida y evitar conflictos al leer columnas. Los kernels elemento a elemento (GELU, residuales, Adam) usan bloques unidimensionales de 256 hilos con un hilo por elemento y accesos perfectamente coalescidos, ya que están limitados por ancho de banda y no por cómputo. Cada lanzamiento va seguido de `cudaGetLastError()`; compilando con `-DVIT_DEBUG` se añade además `cudaDeviceSynchronize()` para localizar errores asíncronos de ejecución.

Un detalle relevante para interpretar el experimento de memoria de la Sección 5.2: el kernel de QK^T y el de $P \cdot V$ tienen la dimensión contraída K en lugares distintos. En QK^T , $K = D_h = 16$ (exactamente el ancho de un tile), así que el bucle de tiling ejecuta *una sola iteración*. En $P \cdot V$, en cambio, $K = T$ (hasta 50), así que ese mismo bucle necesita hasta $\lceil 50/16 \rceil = 4$ iteraciones, cada una con dos barreras `__syncthreads()`. Esta diferencia estructural resulta decisiva para explicar por qué el tiling beneficia a un kernel y perjudica al otro.

4.5. Entorno y protocolo de medición

El entrenamiento se ejecuta en Google Colab con GPU NVIDIA T4 (arquitectura Turing, `sm_75`), compilando con `nvcc -O3`. Los tiempos se miden con `cudaEvent_t`, que registra tiempo de GPU y no de CPU, lo correcto para kernels asíncronos. El microbenchmark del experimento 2 descarta iteraciones de calentamiento (la primera invocación incluye la carga del módulo y la compilación JIT del PTX), promedia sobre 200 repeticiones dentro de una única ventana de medición para que la latencia de lanzamiento no domine, y reporta la mediana de 5 ventanas para descartar valores atípicos causados por otros procesos en la GPU compartida. Para el tiempo por época *extremo a extremo* con memoria global (Sección 5.2), se corrieron solo 3 épocas en vez de 15: ese experimento mide únicamente velocidad, no precisión final, así que entrenar hasta converger habría gastado presupuesto de GPU sin aportar nada a la comparación. Por eso la precisión de esas corridas no se reporta ni se compara con la Tabla 1.

4.6. Validación de la corrección

Antes de dar por buena cualquier medición se ejecutan dos controles independientes. El primero (`tests/verify_kernels.cpp`) emula en CPU las mismas expresiones de índice de los kernels tileados (misma estructura de bloques, hilos y tiles) y las contrasta contra una implementación ingenua, incluyendo tamaños que no son múltiplos de 16 para ejercitar las guardas de frontera; además compara cada gradiente analítico contra su aproximación por diferencias centradas. El segundo es una réplica exacta de la arquitectura en PyTorch (`scripts/vit_reference.py`) que fija la precisión esperada: si la versión CUDA queda muy por debajo, el fallo está en los kernels y no en los hiperparámetros. Sobre el mismo subconjunto y con los mismos hiperparámetros, esa referencia (ejecutada en CPU/MPS, no en la T4) alcanza 87,35 %, 89,05 % y 91,30 % de precisión en test para

parches de 4×4 , 7×7 y 14×14 respectivamente, de modo que la implementación en CUDA debe situarse en ese entorno. No se espera una coincidencia exacta, porque el dispositivo, la inicialización aleatoria y el orden de acumulación en punto flotante difieren. La misma comprobación sirvió para fijar el learning rate: con $3 \cdot 10^{-4}$ las tres configuraciones quedaban sin converger a las 15 épocas, con lo que la comparación habría medido velocidad de convergencia en lugar de calidad del modelo. Por último, el propio microbenchmark comprueba que las versiones global y compartida produzcan resultados idénticos antes de compararlas: un *speedup* sobre un kernel incorrecto no significa nada.

5. Resultados

5.1. Experimento 1: número de parches

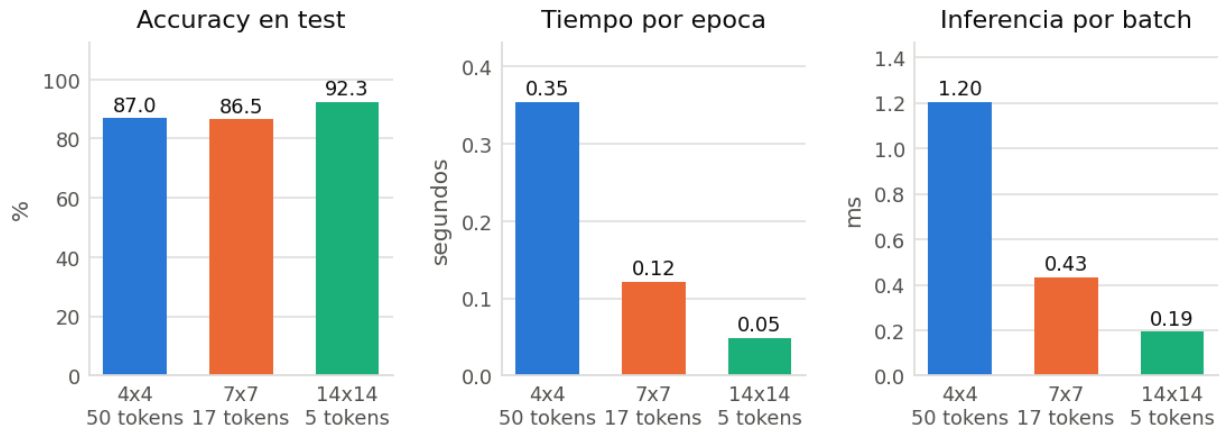


Figura 1: Precisión en test, tiempo por época y tiempo de inferencia por batch para los tres tamaños de parche. Menos parches implica menos tokens y, por tanto, menos cómputo en la atención.

Tabla 1: Comparación completa por tamaño de parche. Subconjunto de 5 000 imágenes de entrenamiento y 2 000 de test, 15 épocas, batch 64, AdamW con learning rate 10^{-3} .

Parche	Tokens	Parametros	Acc. test (%)	Total (s)	s/epoca (s)	Infer. (ms/batch)	Mem. GPU (MiB)
4×4	50	72 074	86.95	5.3	0.35	1.20	112
7×7	17	72 074	86.45	1.8	0.12	0.43	24
14×14	5	80 714	92.30	0.7	0.05	0.19	6

La Tabla 1 y la Figura 1 resumen el experimento. En **coste**, el resultado es exactamente el que anticipa la teoría y es estrictamente monótono en el número de tokens: a más parches, más caro, en los tres indicadores a la vez (tiempo por época, tiempo de inferencia y memoria de GPU). El parche 4×4 (50 tokens) tarda 7.2 veces más por época que el 14×14 (5 tokens), consistente con el coste $O(T^2)$ de la auto-atención.

En **precisión**, en cambio, el resultado *no* es monótono. La configuración 14×14 (5 tokens) gana con claridad, 92.30 %, muy por encima de las otras dos. Pero entre las dos restantes el orden se invierte respecto del número de tokens: el parche 4×4 (más tokens, 86.95 %) supera ligeramente al 7×7 (menos tokens, 86.45 %), una diferencia de apenas medio punto porcentual, dentro de lo esperable como ruido de una sola corrida con una única semilla aleatoria (ver Limitaciones). La lectura honesta no es “menos parches siempre gana”, sino que **la configuración extrema de 5 tokens gana por un margen claro y sistemático** (5.85 puntos sobre la peor de las otras dos, visible ya desde las primeras épocas en la Figura 2), mientras que la diferencia entre 17 y 50 tokens no permite concluir nada con un solo experimento por configuración.

Para el caso claro (por qué 5 tokens gana), la explicación tiene tres partes. Primero, los dígitos de MNIST son imágenes de 28×28 centradas y de trazo grueso: un parche de 14×14 ya cubre un cuadrante completo del dígito, y la información discriminativa (la topología del trazo) sobrevive a esa granularidad. Segundo, con 50 tokens el modelo debe aprender 50 embeddings posicionales a partir de solo 5 000 ejemplos, lo que agrava el problema de optimización sin aportar capacidad útil. Tercero, el ViT carece del sesgo inductivo de localidad de una convolución: debe *aprender* qué parches son vecinos, tarea que se vuelve más difícil cuanto más larga es la secuencia. Estos tres factores explican por qué la secuencia muy corta gana con claridad, pero no predicen un orden fino entre 17 y 50 tokens; de hecho, los datos tampoco lo muestran.

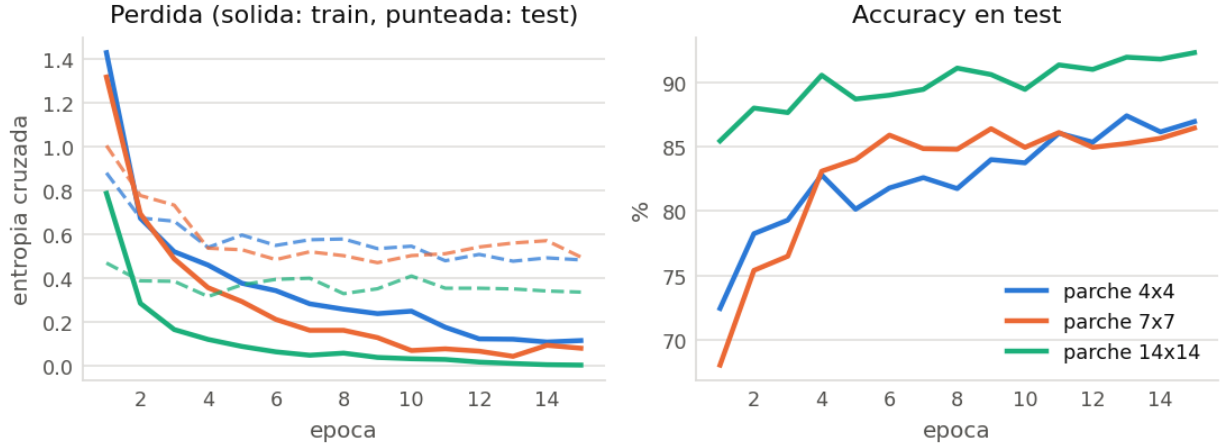


Figura 2: Curvas por época. A la izquierda, entropía cruzada en entrenamiento (línea sólida) y en test (punteada); a la derecha, precisión en test. El eje Y del panel derecho **no arranca en 0**: se recorta alrededor del rango de datos observado (68–92 %) para que las diferencias entre curvas sean visibles, así que las distancias verticales entre líneas no deben leerse en proporción.

Las curvas de la Figura 2 muestran además que la configuración de parche grande no solo termina mejor, sino que *converge antes*: la brecha entre la pérdida de entrenamiento y la de test indica sobreajuste creciente en las últimas épocas, esperable con un subconjunto de 5 000 imágenes y sin aumento de datos ni *dropout*.

5.2. Experimento 2: memoria global frente a memoria compartida

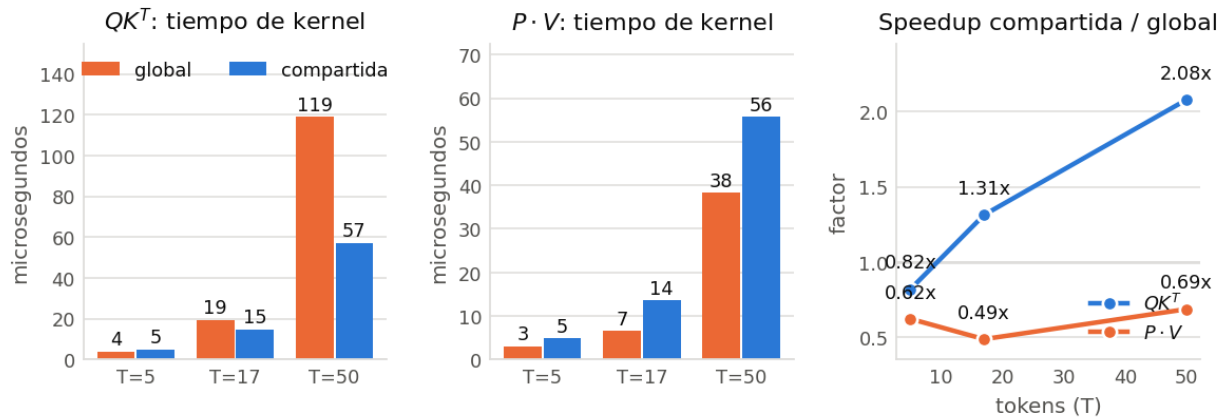


Figura 3: Tiempo de los dos kernels críticos de la atención en sus versiones con memoria global y con tiling en memoria compartida, y *speedup* resultante en función del número de tokens. En el panel derecho, la línea horizontal en 1,0× marca la ausencia de cambio: por encima, la versión con tiling gana; por debajo, pierde frente a la versión con memoria global.

Tabla 2: Microbenchmark aislado de los kernels QK^T y $P \cdot V$. Mediana de 5 ventanas de 200 repeticiones, con calentamiento descartado. Ambas versiones producen resultados bit a bit idénticos (diferencia máxima 0.0 en valor absoluto), así que la diferencia de tiempo no se debe a un error de cómputo.

Kernel	Parche	T	Global (μs)	Compartida (μs)	Speedup	GFLOP/s (compartida)
$P \cdot V$	4×4	50	38.2	55.8	$0.69\times$	367.2
$P \cdot V$	7×7	17	6.6	13.6	$0.49\times$	174.4
$P \cdot V$	14×14	5	3.0	4.8	$0.62\times$	42.3
QK^T	4×4	50	119.1	57.3	$2.08\times$	357.6
QK^T	7×7	17	19.3	14.7	$1.31\times$	160.7
QK^T	14×14	5	4.0	4.9	$0.82\times$	41.5

El resultado no es el que se suele asumir por defecto, y es el hallazgo más interesante del trabajo: **el tiling en memoria compartida no beneficia a los dos kernels por igual**. QK^T sí gana con el tiling, y la ganancia crece con el número de tokens tal como predice la teoría: a 50 tokens el factor es $2.08\times$, cayendo a apenas $0.82\times$ (es decir, la versión con tiling queda *más lenta*) con solo 5 tokens. $P \cdot V$, en cambio, **es más lento con memoria compartida en las tres configuraciones probadas** ($0.49\times$ a $0.69\times$, y sin relación monótona con T : el peor caso no ocurre en ningún extremo, sino en $T = 17$). Ambas versiones producen resultados idénticos bit a bit (columna `max_abs_diff` de la Tabla 2), así que la diferencia no es un error de los kernels: es un efecto real de cómo cada uno usa la memoria.

La explicación está en dónde cae la dimensión contraída K de cada producto, como se adelantó en la Sección 4.4¹. En QK^T , $K = D_h = 16$: el bucle de tiling ejecuta una única iteración, con una sola ronda de sincronización, y cada elemento de Q y de K se reutiliza hasta T veces (una por cada fila o columna de la matriz de *scores* de $T \times T$): la ganancia es real y crece con T porque crece la reutilización. En $P \cdot V$, $K = T$: para $T = 50$ el bucle necesita $\lceil 50/16 \rceil = 4$ iteraciones, cada una con dos barreras `__syncthreads()`, es decir, hasta 8 sincronizaciones de bloque completo por cada elemento de salida. La reutilización potencial también existe aquí (cada fila de P se comparte entre los $D_h = 16$ hilos que calculan las columnas de salida), pero el volumen de datos en juego es minúsculo: la matriz P completa pesa como mucho $50 \times 50 \times 4 \text{ B} \approx 10 \text{ KB}$, muy por debajo de la caché L2 de la T4, así que la mayoría de las lecturas “redundantes” de la versión global ya son aciertos de caché casi tan rápidos como una lectura de memoria compartida. En ese régimen, el costo fijo de las sincronizaciones adicionales de la versión con tiling pesa más que el ahorro de ancho de banda que en teoría debería aportar.

Esto también explica el patrón de la Figura 3: la curva de QK^T sube con T porque su única sincronización se amortiza sobre cada vez más reutilización; la de $P \cdot V$ se mantiene por debajo de $1.0\times$ porque el número de sincronizaciones *también* crece con T , cancelando buena parte de la ganancia que el tiling debería aportar. El efecto neto sobre el entrenamiento completo, donde ambos kernels se ejecutan uno tras otro en cada paso, es pequeño y a veces se cancela: comparando el tiempo por época de principio a fin (Tabla 1 frente a los mismos experimentos con memoria global), la versión con tiling resulta más rápida en la configuración de 5 tokens (hasta $1.14\times$) pero prácticamente empatada en las otras dos (diferencias por debajo del 2 %), porque ahí la pérdida de $P \cdot V$ absorbe casi toda la ganancia de QK^T .

6. Conclusiones

Sobre el tamaño de parche. En MNIST con un presupuesto reducido de datos, el efecto sobre el **coste** es limpio y monótono: menos parches es siempre más barato, con una diferencia de $7.2\times$ en tiempo por época entre los extremos, tal como predice el coste $O(T^2)$ de la atención. El efecto sobre

¹Sección “Diseño de los kernels”.

la **precisión** es más matizado: la configuración de 5 tokens (14×14) gana con claridad (92.30 %, entre 5 y 6 puntos por encima de las otras dos), pero entre 17 y 50 tokens la diferencia es de medio punto porcentual, indistinguible del ruido con una sola corrida por configuración. La lección honesta no es que “menos parches siempre gane”, sino que **el extremo de secuencia muy corta gana con claridad**, probablemente porque los dígitos de MNIST son de trazo grueso y centrados, de modo que la resolución adicional de más parches no aporta información discriminativa mientras sí agrava el problema de aprender embeddings posicionales y relaciones espaciales con pocos datos. Es razonable esperar que el compromiso se invierta en imágenes de mayor resolución o con objetos descentrados, donde el detalle fino sí sea necesario.

Sobre la jerarquía de memoria. Este es el resultado menos esperado del trabajo: el tiling en memoria compartida **no benefició a los dos kernels por igual**. QK^T ganó hasta $2.08\times$, con una ganancia que crece con el número de tokens como predice la teoría. $P \cdot V$, en cambio, resultó *más lento* con tiling en las tres configuraciones probadas (entre $0.49\times$ y $0.69\times$, sin relación monótona con T). Ambos kernels calculan exactamente la misma aritmética con exactamente el mismo tamaño de tile, así que la diferencia no está en un error de implementación sino en la estructura del problema: QK^T tiene su dimensión contraída fija en $D_h = 16$ (una sola iteración de tiling, una sola sincronización), mientras que la de $P \cdot V$ es el número de tokens T (hasta 4 iteraciones y 8 sincronizaciones para $T = 50$). Con matrices tan pequeñas que caben cómodas en la caché L2 de la GPU, el costo fijo de esas sincronizaciones adicionales superó el ahorro de ancho de banda que el tiling debería aportar. La lección práctica, y la más transferible del trabajo, es que **el tiling en memoria compartida no es una optimización universal**: conviene medir cada kernel por separado antes de asumir que la técnica siempre ayuda, porque su beneficio depende de cuántas iteraciones necesita el bucle de reducción y de si el problema es lo bastante grande como para que el tráfico a memoria global, y no la sincronización, sea el cuello de botella real.

Limitaciones. Los resultados se obtuvieron con 5 000 imágenes de entrenamiento, 15 épocas y **una sola semilla aleatoria por configuración**, lo que impide reportar desviación estándar y hace que diferencias pequeñas (como la de medio punto entre 17 y 50 tokens) no puedan distinguirse del ruido de inicialización. Las precisiones absolutas (≈ 90 %) están muy por debajo del 99 % que alcanzan modelos convolucionales sobre MNIST completo; la comparación *entre* configuraciones sigue siendo válida porque todas comparten presupuesto. No se implementó *dropout* ni aumento de datos, y los kernels usan un tamaño de tile fijo de 16×16 sin ajustar por configuración. El entrenamiento es en precisión simple, sin aprovechar los *tensor cores* de la T4. Por último, la explicación del resultado de memoria compartida (Sección 5.2) es mecánica: se apoya en contar iteraciones de tiling y tamaños de matriz, pero no está confirmada con un perfilador; no se dispuso de `nsys` o Nsight Compute en el entorno de Colab usado para separar directamente tiempo de cómputo, tiempo de memoria y tiempo de sincronización.

Trabajo futuro. Tres líneas se derivan directamente de las limitaciones anteriores: repetir cada corrida con varias semillas para confirmar si la diferencia entre 17 y 50 tokens es señal o ruido; perfilar los kernels con Nsight Compute para confirmar o refutar la explicación propuesta para $P \cdot V$; y escalar al conjunto completo de 60 000 imágenes para separar el efecto del tamaño de parche del efecto del tamaño de muestra. Más allá de eso, un tile más pequeño (por ejemplo 8×8) podría ayudar en $T = 5$, donde el tile actual excede el tamaño del problema, y una fusión al estilo FlashAttention (Sección 1) permitiría comparar los kernels propios contra cuBLAS con la técnica ya integrada.

Referencias

- [1] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

- [2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [3] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.
- [4] D. Hendrycks and K. Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- [5] D. B. Kirk and W. Hwu. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 3 edition, 2016.
- [6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [7] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [8] NVIDIA Corporation. Cuda c++ programming guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>, 2024.
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [10] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T. Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning (ICML)*, 2020.